

Batcher: One Engine for Every Scale, Every Shape of Data, and Every AI Workload

A Mergeable Algebra that Unifies Batch and Streaming, from a Core to a Cluster

Stephen Offer

Anyscale • stephen.offer@anyscale.com

Abstract

The data stack most companies run was designed between 2009 and 2014, on assumptions that were all true then and have all since inverted. A single instance now carries 96 to 192 cores and terabytes of memory. Change data capture removed the batch window. Object storage and schema drift removed the catalog, and the operators that replaced it (regexes, JSON paths, Python callbacks, models) carry no statistics at all. Machine-generated documents, images, audio and video became most of the corpus, and a model stopped consuming the finished dataset and started producing intermediate columns inside it, across four tiers at once. A pipeline is now I/O, then CPU decode, then GPU inference, then a CPU join. And it is written in Python.

The engines did not move with any of it. They grew a second execution model for streaming, kept pricing plans from numbers nobody measured, left machine learning to a second system holding a second copy of the data, scheduled one kind of core, and offered an opaque per-row callback for anything SQL could not say. So a team is asked the same question over and over. Pick a scale, pick batch or streaming, pick a shape of data, pick a tier of model, pick a device, pick a language: every answer is a different product, with glue in between.

Batcher is an engine designed after all seven shifts, and one constraint does most of the answering. **One algebra, every scale:** a stateful operator may exist only once, as a mergeable triple $\text{partial} \rightarrow \text{combine} \rightarrow \text{finalize}$ in Rust over Apache Arrow, so one core, ninety-six cores and a cluster differ in scheduling and in nothing else. Small queries pay nothing for distribution; large ones need no rewrite. **One execution model for batch and streaming:** that same triple *is* the incremental form, so a finite table is a stream that ends, and both run under one memory bound that does not change with the source. **Every shape of data, every tier of model:** 30 file and table formats across structured, semistructured, unstructured and multimodal data, and 125 ML entry points from generalized linear models through GPU inference to LLM batch generation. Image and audio decode, mel-spectrograms, embeddings, vector search and inference are expressions over the same Arrow batches under the same optimizer, so a predicate can be pushed beneath a JPEG decode and a tensor never leaves the engine. **An engine that learns:** Batcher measures what every operator did and plans the next query from that evidence, not from vendor constants, under a rule that makes every learned value provably unable to change an answer. **One scheduler across CPU and GPU:** a

plan is split at its resource boundaries and the stages overlap over credit-bounded transport, so the accelerator is not idle while a JPEG is unpacked. **A Python surface the optimizer can see through:** the expression API lowers to the same IR as SQL and executes in Rust, and a callback, when one is unavoidable, is handed a whole Arrow batch.

No axis is traded for another. On relational work, one 96-core machine matches DuckDB on TPC-H [33] sf1, leads Polars and Daft by 1.8–2.2 $\times$, sweeps PyArrow’s Acero engine, and wins 43 of 43 ClickBench queries against DuckDB on identical input. On eight nodes, aggregate shuffle throughput rises superlinearly, 7.6 $\times$ for 4 $\times$ the hardware. On the AI pipelines a purpose-built runtime was designed for, it holds 2–3.5 $\times$ against Ray Data while treating decode and inference as relational operators, and its in-engine mel-spectrogram and MFCC match `torchaudio` to 10⁻⁶ with no equivalent inside any other engine we surveyed. Streaming inherits the batch operators unchanged: peak memory on a join-heavy query is 3.4 MB at one million rows and 3.3 MB at four million, against 198 MB and 794 MB for Batcher’s own materializing executor on the same query.

1 Seven shifts since Spark

Every architecture encodes the world it was born into. The distributed data stack most companies still run was designed between 2009 and 2014, and it was designed well: the assumptions underneath it were all true at the time. Seven of them have since stopped being true, and none of them gradually. Each flipped hard enough to invert the design decision it justified. This paper is organized around them. Six pillars answer the seven, because shifts four and five are two halves of one problem.

Shift one: the machine inverted. In 2012 a large server had 16 cores and 64 GB, so “the data will not fit on one machine” was simply a fact, and everything followed from it: a cluster you start before you ask a question, a scheduler between you and your data, serialization on every hop. A single cloud instance now has 96 to 192 cores and up to twelve terabytes of memory. The workload that needed a rack fits in a footnote. The tax stayed anyway, which is why a query that would take twelve milliseconds in-process still takes seconds through a distributed runtime. *Distribution stopped being a property of the engine and became a property of the query* (Section 2).

Shift two: data stopped arriving in batches. The batch window was a real thing: data landed as a nightly dump, ETL

ran against yesterday, and “daily” was an acceptable freshness target for most of the business. Change data capture, event logs and continuously landing object storage removed the window. The engines of that era answered by bolting on a second execution model beside the first, which means two sets of operator semantics to hold in agreement and a rewrite for any pipeline that crosses between them. *Continuous arrival stopped being a special case and became the default* (Section 3).

Shift three: neither the data nor the transformations can be known before the query runs. Classical optimizers were built for curated schemas, statistics refreshed on a schedule, and queries written once and run for years. That world had ANALYZE jobs and DBAs. This one has object storage, schema drift and JSON columns, and the same query never sees the same distribution twice.

The second half is newer and worse. The operators themselves became opaque. A regular expression, a JSON extraction, a Python callback, an embedding model, an LLM call: none of these has a catalog entry, a selectivity, or a cost anybody has measured. Static cost models price them by inventing a number, and the numbers are not close. A filter assumed to keep a tenth of its rows may keep all of them, and a model call priced like a comparison is off by six orders of magnitude. As shifts five and seven push exactly these operators into the middle of every plan, the fraction of a query the optimizer can honestly price keeps falling. *Optimizing once, from numbers nobody measured, stopped being a performance choice and became a guess* (Section 4).

Shift four: the data became unstructured, and far larger. Analyst projections put the unstructured fraction of enterprise data at 80 to 90 percent [13, 27], a forecast and not a census, and the share is the least interesting part of it. Documents, logs, images, audio, video and sensor traces are produced continuously by machines, not periodically by people, so the volume and the rate rose together, and none of it arrives in columns. An engine whose type system stops at scalars can only treat this as an opaque blob to hand somewhere else. *Most of the data stopped being representable in the engine that was supposed to process it* (Section 5).

Shift five: AI moved from the end of the pipeline into the middle of it. This is the shift we think is most often missed. When machine learning consumed a finished dataset, keeping it outside the engine cost only a file handoff. It no longer consumes the output; it *produces* intermediate columns. Embedding a document, classifying an image, extracting fields with an LLM, scoring a row: each is a transformation with a join or a filter or an aggregate on both sides of it. The variety compounds the problem, because one team now runs generalized linear models, gradient-boosted trees, deep vision models, embedding models and LLMs against the same tables, and each tier used to be a separate framework. *A model stopped being a consumer of the pipeline and became an operator inside it* (Section 5).

Shift six: compute stopped being homogeneous. The 2010s engine scheduled one resource, and a core was a core. A modern pipeline reads compressed bytes, decodes them on CPU, runs a model on GPU, and joins the result back on CPU, so the units are no longer interchangeable and the expensive one is

idle whenever the cheap one is behind. Sizing that by hand is the reason so much GPU capacity runs half-used. *Utilization stopped being about parallelism and became about overlap across unlike devices* (Section 6).

Shift seven: the interface moved from SQL to Python. SQL did not go away and should not, but it stopped being where pipelines are written. The practitioner is a Python programmer, the surrounding code is Python, and the model is a Python object. Engines answered with a user-defined-function escape hatch, which the optimizer cannot see into and which usually runs a row at a time in an interpreter. That turns the natural way to express a transformation into the slowest way to run one. *The default authoring language stopped being the one the engine optimizes* (Section 7).

Each shift dissolved a boundary that used to be an architectural seam and is now a line one job routinely crosses: single-node and distributed, batch and streaming, structured and unstructured, query and model, CPU and GPU, SQL and Python. An engine built around either side of any of them must treat the other as foreign, and Table 1 is the bill for that.

Six pillars pay it off, in order. One algebra spans a core to a cluster (Section 2). One set of operators serves batch and streaming (Section 3). One optimizer plans from measurement (Section 4). One expression surface covers every data shape and model tier (Section 5), one scheduler spans CPU and GPU (Section 6), and one Python surface stays transparent to the optimizer (Section 7).

2 Pillar one: one algebra, every scale

Shift one: the machine inverted, so distribution should be a decision and not an architecture. An engine that assumes the cluster charges every small query for it; an engine that assumes the box cannot answer a large one. Both assumptions are now wrong often enough that neither is a safe default, which means the choice has to move out of the engine and into the query.

That is hard for a reason worth stating plainly. Ask an engine builder how a GROUP BY works and you get two answers. On one machine it is a hash table. On a cluster it is a shuffle, a partial pass, and a reduce. The two share a name, a SQL semantics, and not a line of code.

That is the root of the bifurcation. Two implementations means two performance profiles and two sets of bugs. Eventually it means two different answers. A float key that hashes one way locally and another after a shuffle splits one group in two, and the single-node test suite stays green because the single-node implementation is correct.

Batcher makes that split impossible. A stateful operator may exist only as a mergeable triple, and the three execution regimes vary only its scheduling (Figure 1). One core folds a stream of batches. Many cores partition and merge. Many machines partition, run the partial pass per node, ship pre-aggregated state, and combine in a tree. Formally, for any partitioning $\{p_k\}$ of an input P ,

$$\text{combine_finalize}(\bigcup_k \text{partial}(p_k)) = \text{finalize}(\text{partial}(P)), \quad (1)$$

Shift	The 2010s assumption	What changed	Batcher's answer
One: the machine inverted	One machine is too small, so distribute everything and pay the tax always.	A single node has 96–192 cores and terabytes of RAM.	One mergeable algebra, so distribution is a per-query scheduling decision: no tax on the small case, no rewrite on the large one.
Two: arrival became continuous	Data lands in a nightly window, so batch is the model and streaming is a separate product.	Change data capture and event logs removed the window.	The mergeable triple <i>is</i> the incremental form, so one set of operators serves both under one memory bound.
Three: nothing can be priced in advance	Optimize once, before execution, from catalog statistics.	Schemas drift, and the new operators (regexes, JSON paths, callbacks, models) have no statistics at all.	Plan from what was <i>measured</i> : sketches built during execution, cost coefficients refit from runtime, strategy chosen by a bandit over outcomes.
	Memory limits are a static configuration an operator tunes by hand.	Workloads mix wide strings, embeddings and images, so one setting cannot fit them.	Learn each operator family's footprint as a per-row ratio, and size admission, spill and batch width from it.
Four: the data became unstructured, and larger	Structured data is the data; everything else is an archive.	Machines emit logs, media and sensor traces continuously, and that is now most of the corpus.	Arrow-native decode, resample, crop and tensor kernels in the data plane, so unstructured input is a column and not a blob.
Five: AI moved into the middle of the pipeline	Analytics and ML are separate systems joined by files; each model tier gets its own framework.	A model produces intermediate columns, with joins and filters on both sides, across four tiers at once.	Embed, search and infer as expressions in the same optimized plan: four data shapes by four model tiers, one optimizer (Figure 4).
Six: compute stopped being homogeneous	A core is a core, and the scheduler has one resource to hand out.	A pipeline is I/O, then CPU decode, then GPU inference, then CPU join.	Split the plan at resource boundaries and overlap the stages over credit-bounded transport, with session-warm models and fractional GPUs (Section 6).
Seven: the interface moved to Python	SQL is the language; anything else is an opaque UDF.	Pipelines are written in Python, and the model is a Python object.	A Python expression surface that lowers to the same IR as SQL, plus batch-at-a-time UDFs when a callback is unavoidable (Section 7).

Table 1: Why a new architecture, and not a faster implementation of the old one. Each row pairs a design decision the previous generation made correctly for its era with the shift that inverted it. Shift three carries two rows because it changed both where cardinality estimates come from and where memory budgets come from.

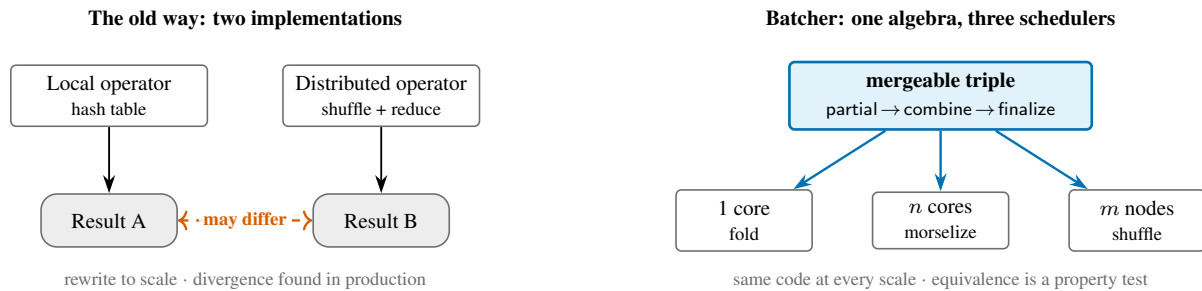


Figure 1: The decision the rest of the system is built on. Batcher admits an operator only if it can be written as a mergeable triple, a partial pass, an associative and commutative combine, and a finalize, and then varies only the *scheduling*. Aggregation, distinct, join build, window, and top- N all qualify. There is no second implementation to disagree with the first, so scaling up is a configuration change, not a migration.

which is a property test, not a comment: a generator produces random partitionings and asserts equality against the single-pass result, and a separate test asserts that a distributed run and a single-node run of the same plan return identical rows.

The paper’s thesis follows from that. **There is no distribution tax on the small case**, because distribution is a scheduling decision made per query, not a runtime you start. **And there is no rewrite on the way out**, because the operator that wins on one core is the operator that runs on eight nodes. Since mappers publish state that is already already reduced rather than raw, per-node memory stays bounded however wide the cluster grows, so shuffle throughput rises with node count and does not collapse into contention (Figure 6).

The triple itself is old: Gray et al.’s algebraic aggregate [12] and MapReduce’s combiner [7]. Every distributed engine reaches for it somewhere, usually for aggregation. What is different is treating it as an *admission criterion* for every stateful operator, join build, window and top- N included, so that no second implementation ever exists to drift from the first. The cost is real and we pay it: an algorithm with no mergeable form does not get added, and Section 13 names the operator where that still hurts.

3 Pillar two: batch and streaming are one execution model

Shift two: arrival became continuous, so a streaming query should not be a different program. The batch window is gone, but the engines built around it kept their shape and grew a second execution model beside the first. That is usually presented as a feature, not an accident. The cost lands in three places: every operator has to be implemented and maintained twice, the two copies have to be proven to agree, and any pipeline that crosses between them has to be rewritten. That is every pipeline that succeeds, because freshness requirements only ever tighten.

The same constraint that settles the scale split settles this one.

Batcher reconciles them lower down. Its executor is a streaming executor for *every* query. Operators pull small batches through a pipeline and materialize only at breakers, so peak memory is the breakers’ state plus one batch per worker: a constant, independent of how much data arrives. On a join-heavy shape that is 3.4 MB at one million rows and 3.3 MB at four million, where Batcher’s own materializing executor grows from 198 MB to 794 MB on the same query. The bounded case is not a special mode. A finite table is simply a stream that ends.

That matters more than it sounds, because the mergeable triple *is* the incremental form. An aggregate that folds each batch through partial and combine never reads its whole input, which is exactly what a continuous query needs: state that updates per arrival and finalizes on demand. The operator a batch query uses and the operator a continuous query needs are not two implementations that happen to agree. They are one implementation, and the difference is whether anything ever calls finalize. The same property makes an operator a pipeline breaker for *ordering* without making it a materialization point for *memory*, which is why a streaming aggregation over an unbounded

source has a memory bound at all.

The practical payoff is that the memory story does not change with the source. A query over a hundred terabytes of Parquet and the same query over a Kafka topic run the same operators under the same bound, and a pipeline that outgrows its batch window does not get rewritten to survive.

4 Pillar three: an engine that learns

Shift three: neither the data nor the operators can be priced before the query runs. Catalog statistics assume a curated schema and a maintenance window, which object storage removed; and no catalog has ever held the selectivity of a regular expression or the cost of an LLM call. An engine facing that has two options. It can keep inventing numbers, which is what a static cost model does, or it can measure itself and plan from what it saw. Everything in this section follows from taking the second option seriously, including the parts that look like over-engineering: if the plan is going to be wrong, the engine needs to find out *during* the query, not after it.

Measuring itself is only sound because of the first pillar. Because the same operator runs everywhere, **a measurement taken anywhere is valid everywhere**: a cardinality observed on one core is evidence about the same operator on ninety-six. Refusing to fork the operator is what licenses the engine to plan from its own history.

Every other engine in Table 2 ships its cost model as constants, chooses join strategy by heuristic, and sizes memory from a plan estimate. Those constants were tuned once, on hardware nobody here is running, against data nobody here has. Batcher replaces each with a measurement: cardinality from sketches built during execution and the true row counts of previous runs, cost coefficients refit from observed operator times, physical strategy from a bandit over measured outcomes, and memory from each operator family’s real footprint.

A single rule keeps this safe. **Every learned value is result-invariant**. Each bandit arm emits the same relation, a threshold picks between equivalent algorithms, a partition count only shards. Learning changes how much memory a query reserves, when it spills, how wide a batch is, and which of several equivalent algorithms runs. It can never change the answer. That is property-tested, and a cold store reproduces the untuned engine byte for byte, so the worst a stale value can do is cost throughput. This is a deliberately different bet from learned optimizers that predict plans [17, 21, 22], which change which plan runs in ways that are hard to bound. A weaker signal buys a far stronger safety property, and it is why the feature ships on by default.

4.1 Three clocks, not one

Adaptive query processing usually means one thing: re-plan at a stage boundary. Batcher does that too. It is the least interesting of the three places it adapts.

Microseconds: the filter reorders itself. A conjunctive filter evaluates its cheapest predicate first, which is what a static cost model can tell you. But cost is not selectivity. The case a static

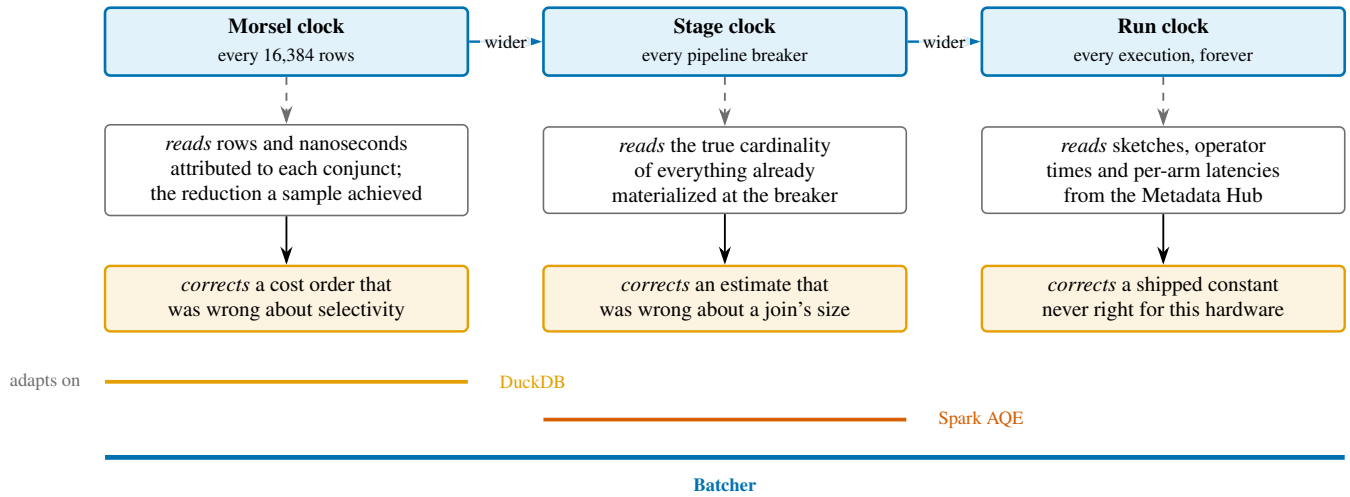


Figure 2: Adaptivity is usually discussed as one feature. It is three, they run at timescales four orders of magnitude apart, and they correct different errors: a cost model that ranked predicates by price and not by selectivity, an estimate that mispriced a join, and a constant compiled in before the hardware existed. Each clock reads only what has already been measured, so none of them is a prediction. The outer clock is the one none of the five comparators has, and it is the reason a query that has run before does not re-derive its plan from cold priors.

model cannot fix even in principle is two predicates of identical cost where only one is selective. Write the broad one first and nothing gets removed, so the selective one runs at full width anyway.

DuckDB solves this by hill-climbing: swap a random adjacent pair, keep the swap if total runtime improved over the next ten batches. Batcher reaches the answer differently. Because the short-circuit evaluates conjuncts one at a time, it can attribute rows and nanoseconds to each conjunct *individually*, and order by time-per-row over fraction-removed. The implied order arrives after a **single morsel** where a hill-climb needs tens of batches to walk the permutation. On the adversarial shape, two regular-expression matches of identical static cost with the broad one written first, that is 48.64 ms down to 28.18 ms. Correctness is free here: the conjuncts of an AND commute, so every order yields the identical mask. Two workers may briefly disagree about the best order at no cost, which is why the shared state needs no lock.

Microseconds: the aggregate picks its own algorithm. A parallel aggregate normally hashes each morsel into its own table and merges the tables. That is right when grouping reduces. It is wrong when grouping does not: `GROUP BY l_orderkey, l_linenummer over TPC-H lineitem` reduces nothing at all, so every partial row survives and the merge inherits the whole relation. The alternative is to partition the input by key first and aggregate each partition exactly once. Measured on a 60M-row table across 96 cores, the two paths cross just under 0.18 rows kept per input row:

rows kept per input row	0.10	0.34	1.00
partial → combine (ms)	125.7	350.9	1351.6
partition → aggregate (ms)	198.2	185.3	370.2

This is more than a tuned threshold, for two reasons. The choice comes from a *measurement* and not an estimate: the

optimizer’s distinct-count for a group key is a sketch, and after a filter it is a guess about a distribution nobody has looked at. So the executor partials a sample of morsels and reads the reduction those partials actually achieved. If grouping does reduce after all, the sample is handed straight back to the reducing path. No work is wasted either way.

Second, look at what the alternative path is. `partition → partial → finalize` is the exact composition Batcher runs *across machines*, executed here across cores. The high-cardinality single-node aggregate is the distributed algorithm, chosen at runtime. This is the constraint paying for itself. Refusing to fork the operator did not just prevent a class of bug. It handed the single-node executor a second strategy for free.

Stages and runs. The outer two clocks are the familiar ones. At a pipeline breaker the engine has already measured what it just processed, so it re-plans the remainder on real cardinalities. Across executions it accumulates sketches, refits cost coefficients and converges bandits. Figure 2 sets the three side by side, and the interesting column is the third: it is the one that makes a query cheaper the second time it runs, and none of the five comparators of Section 14 has it.

5 Pillar four: every shape of data, every tier of model

Shifts four and five: the data stopped being representable and the model stopped being downstream. These arrive together and this pillar answers them together, because they are the two axes of one grid: the *shape* of the data and the *tier* of the model. Both fragmented independently, and a team lands in the cross product: a job that reads Parquet and JPEGs, embeds one and joins the other, then scores the result with a gradient-boosted tree and an LLM in the same pass.

Batcher reads 30 file and table formats and exposes 125 public

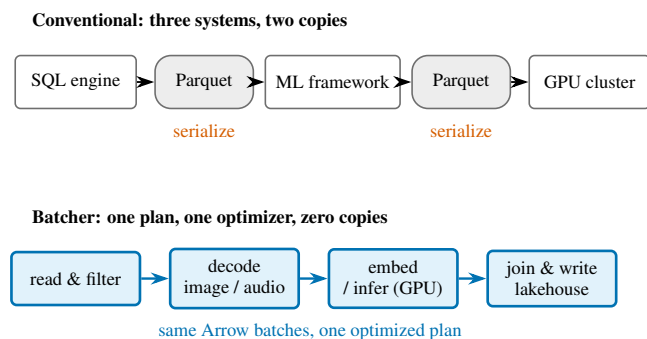


Figure 3: Why the seams cost more than the compute. In the conventional stack each handoff is a full serialize, a write, a read and a deserialize of the whole dataset. No optimizer sees across the boundary. In Batcher the decode, the embedding, and the inference are expressions in the same plan as the filter and the join, so a predicate can be pushed below a decode and a tensor never leaves the engine.

ML entry points. The point of those numbers is not their size but that a single plan spans them: Figure 4 is a four-by-four grid of data shape against model tier, and one optimizer covers all sixteen cells.

Engines specialize by *data shape*. DuckDB and Polars are superb over structured columns and have no image or audio surface at all. Ray Data and Daft handle multimodal data well and have no relational optimizer. Spark covers structured and semistructured, and its multimodal story is a UDF. A team whose job spans columnar files, nested documents, free text and images, which describes most teams, therefore runs several systems and maintains the glue between them.

They specialize by *workload tier* too: classical machine learning in one framework, deep-model inference in another, LLM generation in a third, and none of them is the system holding the data. Batcher’s ML surface spans all three, from generalized linear models and mixtures through GPU inference to LLM batch generation with request packing and structured output. The claim Figure 4 supports is bounded, and worth stating precisely: not that no engine can reach any given cell, but that none of the five comparators reaches all sixteen *natively, in one plan, under one optimizer*.

In Batcher, preparing data for a model is not a different kind of work from querying it. Image decode, crop and normalization to a tensor are Arrow kernels in the data plane, not a Python UDF. So are audio decode, resampling, mel-spectrogram and MFCC, the last two matched to `torchaudio` within 10^{-6} and, across the comparator set of Section 14 (DuckDB, Polars, Spark, Ray Data and Daft, at the versions listed there), without an in-engine equivalent anywhere else. So are vector distances, embedding pooling and fuzzy string matching. Vector search is a plain query. Batch inference runs against pools that load a model once per session and reuse it, with CPU decode and GPU compute overlapped so the accelerator is not idle while a JPEG is unpacked.

For a concrete example of what living inside the engine buys, take epoch shuffling. Permuting a training corpus normally means materializing an index of every sample, hundreds of gigabytes at ten billion of them, and it has to exist before the

first batch is read. Batcher replaces the list with a keyed bijection, a small Feistel network [20] with cycle walking [4], so the epoch order is computed, not stored. Driver memory is constant in corpus size and a mid-epoch resume is arithmetic, the same idea as [25] with world-size-independent ordering.

The consequence is Figure 3. A retrieval pipeline, read a corpus, strip HTML, chunk, embed, write to a lakehouse table, is one plan and one pass over the data. So is a multimodal one: read images from object storage, decode and normalize, run a model, join the predictions against a customer table, then `MERGE` into Delta. The filter that removes 90% of the rows runs *before* the decode, because one optimizer sees both. Others can do each of these things, and Daft in particular has a strong multimodal surface; what is unusual is that they are all expressions in one optimized plan over one columnar format, governed by one optimizer and one memory manager.

6 Pillar five: one scheduler across CPU and GPU

Shift six: compute stopped being homogeneous, so utilization became a question of overlap. A multimodal pipeline is a chain of unlike costs. Reading compressed bytes is I/O, decoding a JPEG or resampling audio is CPU, a forward pass is GPU, and the join that puts the prediction back next to the customer row is CPU again. Run that chain inside one worker and the accelerator idles through every decode. This is the single most common reason a GPU pipeline underperforms its hardware, and the usual remedy is for the user to hand-tune a ratio of CPU workers to GPU workers per stage.

Batcher treats it as a scheduling problem the engine owns. A plan carrying a resource-class boundary is split there: a pool of CPU producers decodes and preprocesses, a pool of GPU consumers runs the model, and Arrow batches move between them over the same credit-bounded Flight transport the shuffle uses, so the window between the stages is bounded and a slow consumer applies backpressure and does not accumulate. Both stages run the identical sub-plan through the ordinary executor; only the placement differs, which is what keeps the split from becoming a second semantics. A chain with no resource boundary to split at falls back to the embarrassingly-parallel map, so a homogeneous pipeline pays nothing for the mechanism.

Two further decisions matter more than they sound. Models load once per *session* and are reused across executions, so a pipeline that runs the same model twice pays for it once; the alternative, respawning per execution, makes model load a per-query tax that dominates short jobs. And a GPU is divisible: several small models can be packed onto one device by fractional allocation, so none of them reserves a whole card. Section 14.6 reports what this is worth against a runtime purpose-built for GPU pipelines: $2\text{--}3.5\times$ across ResNet-50 inference, image and text embeddings, video-clip inference and training-data ingest, with fractional packing at $1.96\times$. What those numbers show is that the GPU stages achieving them are relational operators, under the same optimizer as the joins around them.

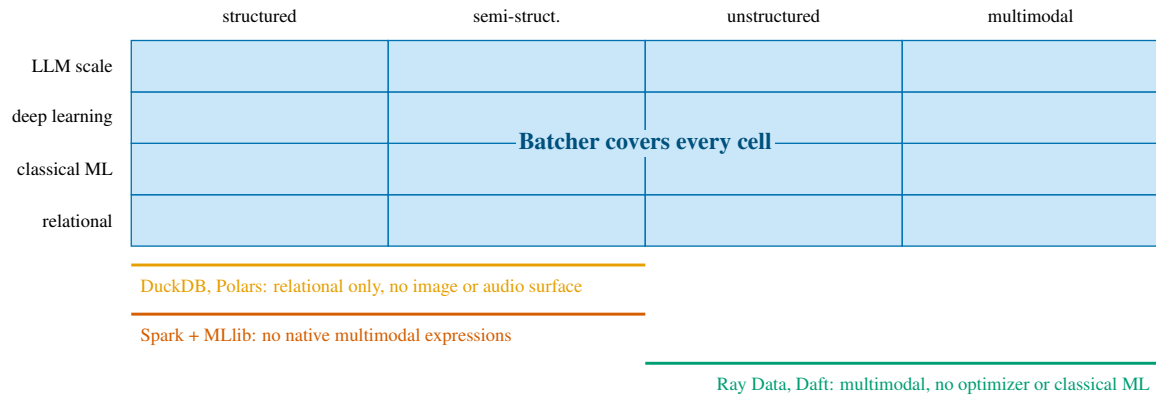


Figure 4: The second fragmentation. Engines specialize by *data shape* across the top and by *workload tier* down the side; the coloured spans show how far each comparator reaches. None of the five covers both axes natively inside one optimized plan. Batcher fills the grid: 30 file and table formats across the four shapes, and 125 public ML entry points from generalized linear models and mixtures through deep-model inference to LLM batch generation.

7 Pillar six: a Python surface the optimizer can see through

Shift seven: the interface moved to Python, so the escape hatch became the main road. The historical bargain was that SQL is restrictive and therefore optimizable, while a user-defined function is expressive and therefore opaque. That bargain fails when the practitioner writes Python by default: the natural way to express a transformation becomes an opaque per-row callback, which the optimizer cannot reorder, cannot push a predicate through, and cannot run anywhere but a Python interpreter. Engines that answer “use a UDF” for anything beyond scalars are answering with their slowest path.

Batcher’s answer has three parts, in order of preference. First, make the UDF unnecessary. The public surface is a Python expression API (`col("x") * col("y")`, `.str.contains(...)`, `.dt.truncate(...)`, `.list.cosine_similarity(...)`, `.image.to_tensor(...)`, `.audio.mel_spectrogram(...)`), and every one of those lowers to the same `bc_expr::Expr` the SQL front-end produces, so it is optimized, JIT-eligible and executed in Rust. Python builds the plan and never touches a row. The breadth of that surface is the whole point of Figure 4: the transformations that would otherwise be UDFs are operators.

Second, when a callback is genuinely required, make it batch-first. A Batcher UDF receives a whole Arrow `RecordBatch` and returns one, so the interpreter is entered once per sixteen thousand rows and not once per row, and a vectorized library or a model call inside it operates on the batch it wants. A class-based UDF keeps its state, whether a loaded model, a tokenizer or a connection, across batches and across executions, which is what makes the session-warm pools of Section 6 possible at all.

Third, keep SQL. It remains the right language for a relational question, so the same engine accepts it through `bt.sql` and a session catalog, parses it to the same `LogicalPlan`, and puts it through the same 375 rules. Vector search is the clearest illustration that the two surfaces meet: `ORDER BY list_cosine_similarity(emb,`

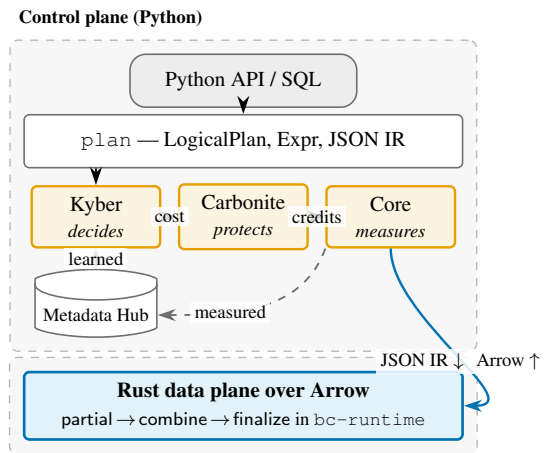


Figure 5: The contract loop. Python builds and optimizes a plan and ships it as JSON IR; all per-row work happens in Rust over Arrow and returns zero-copy. *Core measures*, *Kyber decides*, *Carbonite protects*, and the three cannot import one another, enforced on every commit. That lint is what stops the optimizer from quietly collecting its own telemetry, which compiles and passes tests while corrupting the loop.

[...] `LIMIT k` is an ordinary SQL query, and it runs the same kernel the Python accessor calls. Neither surface is a wrapper over the other, because both lower to one plan.

8 Architecture

Batcher is a Python control plane over a Rust data plane, with Arrow `RecordBatch` the only columnar contract at every internal boundary and across the FFI: roughly 100,000 lines of Rust across 13 crates and 233,000 of Python. Python builds a plan, optimizes it, and ships it as JSON; data returns zero-copy through the Arrow C Data Interface. Python never iterates a row, which lets the control plane be a high-level language while nothing it does sits on a per-row path.

The three subsystems in Figure 5 are isolated by a lint contract that forbids them importing one another. The rule reads as bureaucratic until you see what it prevents: an optimizer pass

that starts collecting its own telemetry, or an executor path that starts making optimization decisions, will compile and pass every test while silently corrupting the feedback loop the design depends on.

Two execution tiers share one expression type: an interpreter that is the correctness oracle, and a compiled fast path that must be bit-for-bit identical on the subset it supports and fall back silently otherwise. Sharing the expression type makes parity checkable; a differential test over both tiers on one input makes it checked. Neither is optional. A divergence here, one tier wrapping an integer overflow where the other raises, is invisible in any single result, because which tier ran depends only on whether the expression happened to compile.

9 How Batcher compares

Three systems set the bar Batcher is measured against: DuckDB single-node, Daft on Arrow, and Spark distributed. Two more bracket the space. **DuckDB** [26] is the single-node bar: vectorized, a compressed native store with zone maps, optimized once before execution, and it does not distribute. **Spark** [1, 35] distributes, and its Adaptive Query Execution [29] re-plans on measured statistics at stage boundaries; its unit of movement is a row-shuffle over JVM serialization, and its fixed per-query cost is large. **Ray Data** [32], on Ray [24], sits in a different category: a streaming runtime for ML pipelines, with actor pools for GPU work and no relational optimizer. We benchmark against it throughout, because a data engine that claims to serve AI workloads should be held to the standard of the runtime those workloads use today. It is a yardstick here, not a rival. **Daft** [9] and **Polars** [34] are native Rust engines over Arrow [30]; Daft distributes over Ray, Polars does not.

Table 2 places Batcher against all of them, and the pattern in it is the paper’s thesis in tabular form. Batcher does not lead because its kernels are faster than everyone else’s, on several rows it is at parity, and on storage it is behind. It leads on the rows where the previous generation shipped a constant and Batcher ships a measurement, and on the rows where the previous generation required a second system.

10 What we took from Spark, and where we depart

Spark’s authors got the hard parts right, and several of its ideas are still doing work here. Treating a streaming query as a batch query over an unbounded table is the correct framing and we adopted it. Adaptive Query Execution was the first widely deployed argument that a plan should change once real cardinalities are known. Whole-stage code generation established that an analytical engine should compile its inner loops, not interpret them. None of what follows says those were mistakes. They were right for the hardware and the workloads of the time. The four shifts of Section 1 moved both, and three concrete departures follow from that.

The unit of movement. Spark moves rows through JVM serialization, which was reasonable when no ecosystem agreed

on a columnar format. Arrow settled that. Batcher moves columnar batches with no serialization step: the memory one operator wrote is the memory the next one reads, and the boundary to Python is a pointer handoff. This is the largest single source of the gap in Table 3, and it is an ecosystem change more than an engineering one.

The cost of being distributed. Spark’s fixed per-query overhead buys fault tolerance and elasticity that matter above a certain size; below it the overhead is pure cost, and shift one moved the threshold. The departure is not that Batcher is cheaper at small scale but that it can *choose*: Spark’s local and distributed paths are the same path, so there is no decision to make.

Where the statistics come from. Catalyst optimizes from catalog statistics and AQE corrects at stage boundaries. Both assume the catalog is a reasonable prior, which shift three removed, and the correction arrives only at a shuffle. Batcher keeps that correction and adds a clock on either side of it (Section 4.1). The one Spark has no equivalent for is the outer clock: a query that has run before should not re-derive its plan from the same cold priors.

11 Kyber: an optimizer that learns

11.1 375 rules, and why that is safe

Seven phases run in a fixed order. The four rewrite phases (normalize, rewrite, pushdown, fusion) iterate to a fixpoint because their rules are confluent; the cost-based and physical phases (join reorder, selection, enforce) run once, because they make a decision and do not converge to one. Measured evidence enters at three of them: cardinality drives join ordering, refit cost coefficients drive fusion, and a bandit drives the physical selection.

Rules are grouped by family, boolean and Kleene algebra, sargability, temporal extraction, predicate inference, join shape, set operations, window canonicalization, and expression families grouped by value domain, and discovered by a decorator registry. A rule set this size is safe only if each rule preserves results and no two interfere, and Kyber proves both mechanically. Each rule carries a plan-shape unit test and a differential test against DuckDB across nulls, empties and the type edges. The set as a whole is property-tested: a generator builds a random table and a random valid pipeline and asserts that the result under all 375 rules equals the result under none, order-independent. The same suite proves confluence and termination, the set converges to a deterministic fixpoint inside the production iteration budget, so the rules provably do not oscillate.

Two invariants bound what the algebraic families may do, and both have bitten. Integer arithmetic wraps on overflow, bit-for-bit with the JIT, so reassociation may only reduce $=$ and $\neq$, bijections of $\mathbb{Z}/2^{64}$, hence wrap-invariant, and never an ordered comparison, since wrapping breaks monotonicity. Three-valued logic means every boolean law is proven over $\{\top, \perp, N\}$; $x \wedge \neg x \rightarrow \perp$ fires only on never-null predicates. Some identities that look safe are refused, each refusal pinned by a test: $\text{pow}(x, 1)$ is not folded, because libm’s `pow` is not correctly rounded and the identity is therefore not provable, while

	Batcher	DuckDB	Polars	Spark	Ray Data	Daft
Single-node / distributed	one algebra	single-node	single-node	two paths	two paths	two paths
Optimizer	375 rules, bushy DP	rules + DP	rules	Catalyst	n/a	rules
Cardinality from	sketches + measured	table stats	estimates	table stats	n/a	estimates
Cost model	refit from runtime	static	static	static	n/a	static
Physical strategy	bandit over past runs	heuristic	heuristic	heuristic	n/a	heuristic
Memory sizing	learned per-row	static	static	static	static	static
Re-optimizes mid-query	stage boundary	no	no	stage boundary (AQE)	n/a	no
In-engine audio / mel	native kernels	none	none	none	UDF	UDF
Vector search in SQL	yes	no	no	no	n/a	no
GPU batch inference	warm pools	none	none	external	actor pools	actor pools

Table 2: Batcher’s competitive set is DuckDB, Daft and Spark; Polars and Ray Data bracket the space, the latter an adjacent streaming ML runtime and not a relational engine, so its relational rows read n/a. Blue marks a capability no other column has; amber marks its absence. The decisive block is the middle: every input to a planning decision derived from a measured run and not shipped as a constant, and a native AI surface inside the same optimized plan. No other engine here has either, and the two are connected — Section 2’s single algebra is what makes a measurement taken on one core valid on ninety-six.

$\text{pow}(x, 0)$ is folded because IEEE 754 specifies it as exactly one for every base. No timezone-conversion rewrite exists at all, a same-zone conversion is not the identity, because the engine nulls DST-ambiguous and nonexistent local times, and dropping the call would resurrect those rows’ values.

11.2 Provenance: rules gated on proof, not confidence

Every cardinality and column statistic carries *where it came from*, and the distinction between proven-exact and estimated-however-confidently is doing real work, not annotation.

Only two rule families fire on EXACT provenance. The first drops a `Distinct` over a column proven unique, skips a `Sort` over at most one row, prunes a constant sort key, and decides a column-to-column comparison from both columns’ proven bounds. The other folds a subtree an exact cardinality proves empty. A sketch estimate may never drive either, because dropping a `Distinct` on a wrong guess is silent data corruption: the query returns quickly, with duplicate rows, and nothing turns red. Everything else in the optimizer may consume an estimate, because the worst outcome there is a slower plan.

There is a boundary here that is easy to cross by accident. Provenance says where a number came from; it says nothing about whether the number was *right*. The gate deciding whether a query pays for stage-by-stage re-optimization used the label as a proxy for “we do not know this size,” which are different statements, and the label is sticky because the one-shot path never records an intermediate operator’s measured cardinality against its signature. A shape can therefore be estimated to within a percent of actual forever and still read as a guess: TPC-H q5’s operands land within $1.0\times$ of actual and carried the default label anyway.

The gate now asks the evidence rather than the label. Core already records, per operator signature, the rows produced alongside the rows predicted, which the optimizer folds into a bounded window of $\log(\text{actual}/\text{estimated})$ to derive its correction factor. A shape is treated as confidently sized when that window holds at least a few observations and *every one of them* sits inside the re-optimization threshold. Reusing that threshold and not invent-

ing one is deliberate: it is the q-error at which the adaptive loop re-plans, so a history that never crossed it is a history in which a stage boundary would never have fired. The maximum is used, not the mean, because alternating $4\times$ over- and under-estimates average to 1.0 and would read as flawless while being exactly the shape a stage boundary exists to correct.

Measured over the 22 TPC-H queries, counting routing decisions and not time so a loaded machine cannot move the number (`adaptive_gate_routing.py` in `benchmarks/internals`): **15 of 22 queries routed to staging on a cold hub, and 0 once their estimates had demonstrably held up**. All 15 still route to staging when the recorded history shows the estimates missing, so the correction suppresses the cost without suppressing the case staging exists for. A cold hub knows nothing and behaves exactly as before, which keeps a first run’s plan unchanged.

11.3 Cardinality: four sources, strict precedence

Estimation starts from Selinger-style selectivities [28] and is superseded in fixed order by exact footer metadata, then by sketches built during execution (HyperLogLog [10] for distinct counts, KLL [15] for quantiles), then by measured cardinalities recorded under a plan signature. Every sketch is mergeable with a fixed seed, so one built per partition merges identically to one built on a single node: the algebra of Section 2 applied to statistics.

Getting this wrong is expensive in a specific way. TPC-H’s `orders.o_orderkey` carries no distinct count in any file footer, and without one the join estimator falls back to assuming a primary-key relationship, which is right for a fact-to-dimension join and catastrophic for a many-to-many key. At sf10 that priced one join at 1.5 million rows where the true output is about six billion, and the executor set about materializing it, passing 110 GB resident. Kyber now measures the distinct count before the optimize that fixes join order for the whole query. Seeded, the planner picks the order DuckDB picks, and that query falls from $8.81\times$ to $3.95\times$ of DuckDB.

11.4 Cost, refit from runtime

Three axes, CPU, I/O, network, collapse to one scalar, with network shuffle bytes weighted at twice local bytes. Per-operator coefficients (inserting into a hash table costs more than probing it) are *recalibrated from measured operator times* once enough samples accumulate, clamped so timing noise cannot produce a degenerate model. Join ordering is dynamic programming over *bushy* plans throughout: exhaustive subset DP at or below 12 tables, a connected-subset (DPccp-style) DP to 20, and a greedy left-deep fallback beyond that or when a dense graph exceeds the DP's work cap.

11.5 Two learned primitives

All of Kyber's self-tuning reduces to two reusable pieces. Both read $O(1)$ sufficient statistics from the hub, keyed by plan signature and scoped to the host's hardware profile, so a value learned on a 16-core laptop never steers a 96-core planner.

A **deterministic UCB1 bandit** [2] selects over a fixed arm set from measured per-arm latencies. There is no RNG and ties break by arm name, which matters more here than in a typical bandit application: a nondeterministic optimizer makes every performance regression unfalsifiable. Arm statistics are a *discounted* Welford state (n, μ, M_2) and not raw sums, so variance is numerically stable and old observations decay as machine and data change. Its callers are the join-strategy choice, hash, broadcast, sort-merge, and the staged-versus-one-shot routing decision below.

An **OLS two-line crossover** learns a *threshold* where the bandit learns a choice. It fits $t \approx a + bx$ per algorithm and solves for the x (build bytes, or build rows) at which the cheaper-below algorithm is overtaken by the cheaper-above one. The result is clamped to a band around the shipped default, so one noisy early fit cannot send a threshold somewhere absurd, and an ambiguous fit returns nothing and the caller keeps the default. Its callers are the broadcast byte threshold and the sort-merge row crossover.

A third layer learns per-signature *scalars*: how big each join side was, how many rows a breaker shuffled, how far an aggregate collapsed its input, whether partial pre-aggregation paid off.

Failure modes. All three layers inherit the result-invariance of Section 4, so the worst a wrong value costs is throughput. Every read is also best-effort: a malformed bucket, a degenerate fit, or a missing key yields the shipped default and never an exception into planning.

11.6 What a learning loop has to test

A learned component can be entirely inert while every result stays correct, and that asymmetry is the one genuinely new hazard this design introduces. Three defects here had that shape: statistics filed under a key that died with the execution, so nothing was ever reused; a calibration epoch counted in a different clock from the refit it tracked, so the plan cache never hit; and a re-optimization gate priced against a control-plane cost the

loop does not actually have. Each produced right answers at every scale while the mechanism that makes plans improve did nothing at all. Fixing the second alone took one sf10 query's steady state from ~ 340 ms to ~ 165 ms.

The consequence for anyone building this is a testing obligation, not a design one. A learning system needs assertions that *the learning happened* (cache hit rates, generation stability, sketch reuse), because a correctness suite passes identically whether the loop is running or switched off.

12 Carbonite: resource decisions from measured footprints

Carbonite decides whether a plan is feasible, hands out memory reservations and shuffle credits, and decides when a query goes out of core. It never rewrites a plan and never computes a result.

12.1 Admission, and a memory model that is a ratio

Conventional admission control answers a boolean: does this plan fit? A rejection is useless to a planner, because it names no smaller thing to try. Carbonite returns the binding constraint *and* a counter-offer, a smaller credit window or a lower parallelism, that Kyber can re-plan around, and it names the *operator* that binds rather than the resource that ran out. This follows from the subsystem split: because Carbonite may not rewrite the plan, the only useful thing it can return is enough information for the layer that may.

Sizing used to come from the plan estimate alone, never from what operators actually used. Two of the choices in fixing that are forced, not tuned.

Fit a per-input-row ratio, not an absolute peak. A family's absolute peak depends on query size, so replaying a stored byte figure onto a differently-sized plan is simply wrong. A bytes-per-row figure is size-general: multiply by this plan's estimated input rows. It is the same reason the cardinality layer learns selectivity, not a row count. One blended figure then feeds every memory decision, including batch sizing, which caps the row count by the widest learned per-row width so a workload of wide rows keeps its true byte working set inside budget where a fixed row count would not.

Fit the 80th percentile, not the median. Memory sizing is one-sided: over-provisioning costs a little headroom, under-provisioning costs the process. A median is by construction exceeded by half the runs, and the runs that exceed it are exactly the large, skewed, wide-row ones that fail. P80 rather than P95 because a family may have ten samples and the tail of a memory distribution is fat and noisy, so chasing a far tail fits noise. The result is clamped to a bounded factor of the plan estimate, and a cold store is a pass-through, so a first run is byte-for-byte the pre-learning behavior.

12.2 Credits, spill, and transport

Flow control is credit-based: one credit is one in-flight batch slot, and a producer blocks at zero, so a channel’s window is a direct bound on its memory. Three layers sit on that. Carbonite grants a static window and clamps any request to a ceiling. A TCP-like controller then grows and shrinks it from observed backpressure, with hysteresis so a channel does not oscillate. And a recurring shuffle warm-starts from the window past runs of that shape converged on. Learning moves only the starting point, so a stale value costs a few round trips of convergence and never correctness.

Spill has two non-obvious properties. **Published shuffle output is memory nobody was reserving:** a mapper hands a bucket to the node’s transport layer and it stays resident until a reducer collects it, so with as many mappers as reducers a node holds its whole share of the shuffle against no reservation. It is now reserved, and it is deliberately the pool’s *first* spill victim, because a published bucket is finished work waiting to be collected. Spilling it stalls nobody, where spilling a half-built hash table interrupts an operator still using it. **And disk gets its own pressure ladder,** measured rather than accounted so a volume filled by a co-tenant is seen as readily as one this query filled. New buckets route to remote storage one level before the floor, because a bucket’s tier is fixed when it opens: by the time the floor is crossed, several are already streaming to a volume that cannot hold them.

Bulk data never enters the Ray object store; it moves over Arrow Flight [31]. Within a shuffle a reducer’s sources fall into three tiers chosen automatically: same-process reads straight from the local store, same-node-different-process memory-maps an Arrow IPC file and decodes zero-copy, and cross-node takes a credit-bounded stream. Measured on a cluster, an eight-producer gather runs at 33.6 GB/s over shared memory against 4.5 GB/s over loopback, and a miss falls back transparently, so single-node equals distributed regardless.

13 Execution: six operator results and two proofs

The executor pulls small batches through linear runs of operators and materializes only at pipeline breakers, in the style of morsel-driven parallelism [18]. Six operator-level results came out of that model, each verified against the sequential oracle before it was timed: a short-circuiting conjunctive filter (1.5–5.7×), dictionary-native comparison (19.6×), a runtime filter sunk across joins (1.9×), a deterministic parallel join map (1.8×), a bounded join fan-out (18× less memory), and a real inequality-join operator (up to 900×). Individually they are unremarkable. Three points about them are not.

Two rewrites that are correct by construction. Most optimizations are justified by testing: apply the fast path, compare against the slow one, ship it when the tests pass. Two here rest on an argument instead, and that is why they can live inside the correctness oracle instead of beside it.

The first short-circuits a conjunctive filter. Arrow has no selection vector, so where DuckDB narrows a predicate chain by masking, Batcher compacts: evaluate the cheapest predicate, gather the survivors, evaluate the rest against a shorter batch. That is only sound if skipping a row cannot hide an error it would have raised, so eligibility is decided by a syntactic test over the expression grammar. Predicates whose failures depend on a row’s *value*, arithmetic that can overflow, a strict cast, a string function, are excluded; predicates whose failures depend only on the *schema* are admitted. The second compares dictionary-encoded columns without decoding them, resting on the observation that an elementwise comparison commutes with a gather. Both produce bit-identical results for reasons that can be stated in a sentence, which is a stronger guarantee than a passing test suite.

The memory bound is asserted, not assumed. Every operator that preserves or reduces rows satisfies “one batch in, one batch out” for free, so Section 3’s constant-memory bound is immediate for almost all of them. A join that *multiplies* rows is the exception, and it takes explicit work: a skewed key can turn each probe row into thousands of output rows, so the operator bounds both its output batches and the index arrays that describe them. On a query where the unbounded version peaked at 13.1 GB, that holds peak memory to 728 MB, an 18× reduction that also cuts runtime threefold with identical results, because the predicate above the join now runs over cache-resident batches.

The bound is an operator-level assertion, not a comment, and that distinction is the transferable part: a correctness suite cannot observe a memory property, so a memory guarantee that is not asserted is not tested.

Where the algebra charges rent. Inequality joins used to lower to a cartesian product with a filter above it, quadratic in time and memory however few rows survive. They are now a real operator implementing IEJoin [16], which removed the wall: a case that took 3.4 seconds at ten thousand rows now takes under four milliseconds, and cases that did not run at all now do.

IEJoin is more general than most inequality joins need, and the difference is worth an operator. It assumes the two conditions constrain *different* right keys, so a left row’s matches are an arbitrary subset that must be read out of a bit array. A *band* — `r.y BETWEEN l.a AND l.b`, and with it interval containment, temporal overlap and range lookup — constrains one key from both sides, so the matches are a contiguous slice of that key sorted once: no union sort, no second axis, no bit array. Both of its bounds are also monotone in the left key, so one merge over the sorted sides replaces a search per row.

That closes the gap for the shape it covers. Against DuckDB reading its own store, the condition for it choosing IE_JOIN over a nested loop, and correctness-gated at every size:

rows per side	500 K	1 M	2 M	5 M
before (IEJoin)	0.69×	0.98×	1.29×	2.24×
after (band)	0.49×	0.55×	0.64×	0.94×

A $2.2\times$ loss at five million rows becomes parity, and the sizes below it become $1.6\text{--}2.0\times$ wins. We call 5 M parity rather than a win because it is the one column that moves with machine load: $0.94\times$ at load average 5, $1.2\times$ at load 14.

14 Evaluation

Setup. One Intel Xeon Platinum 8275CL at 3.00 GHz, 96 vCPU, 184 GiB, Linux 6.8; distributed results on an 8-node Ray cluster, 128 vCPU; GPU results on $8\times$ T4. DuckDB 1.5.5, Polars 1.36.1, Daft 0.7.21, Ray 2.56.0, PySpark 4.2.0, PyArrow 19.0.1. The harness refuses to report a timing for any query whose result does not match DuckDB, compared as an order-independent multiset. Timings are best-of- N warm.

Two measurement hazards are worth naming. A shared machine is not a benchmark machine: at load average 16–41 a repeated run swung $\pm 25\%$, so claims moving less than $2\times$ are quoted from a run at load under 5. And an in-place rebuild of the compiled extension silently replaces the engine mid-benchmark, so we stamp its timestamp around every run and discard any run in which it moved.

14.1 Does the algebra remove the fork?

The central claim is architectural, so its evidence is a testing argument and we state its strength precisely. Equation (1) is property-tested over random partitionings; an integration test asserts single-node equals multi-partition; and an operator matrix crosses {collect, spill, iterate, distributed} with {nulls, empty, one row, duplicates, $-0.0/\text{NaN}$, descending}, because the defects that reach production here are combinations of a non-default flag with a non-default path.

14.2 TPC-H

Table 3 is the headline, and it splits cleanly. Against Polars and Daft, Batcher leads by $1.8\text{--}2.2\times$ in geometric mean at both scales. Against DuckDB the result splits along the line Table 2 predicted: on identical Arrow input Batcher wins 21 of 22 at sf10 and is $1.89\times$ faster overall, and against DuckDB’s compressed native store it loses. q1 and q6 are essentially scans and sit at $1.47\times/1.52\times$, about what reading compressed pages instead of raw Arrow buys. But q21 ($3.18\times$), q9 ($3.79\times$), q7 ($3.19\times$) and q5 ($3.95\times$) are several times that, and those are engine work.

Correctness is not free for everyone. Daft gets TPC-H q6 wrong at both scales, dropping the `l_discount = 0.07` rows because $0.06 + 0.01$ in IEEE double is `0.06999999999999999`; ground truth computed independently in PyArrow equals the official answer, and Polars makes the same mistake. This is why every timing is gated. A benchmark that does not check results reports the fastest wrong answer.

14.3 Operator mix and ClickBench

Table 4 is the cleanest comparison here: one input representation, one machine, no SQL front-end differences, no scan in the timed region for anybody. Batcher sweeps PyArrow’s Acero engine

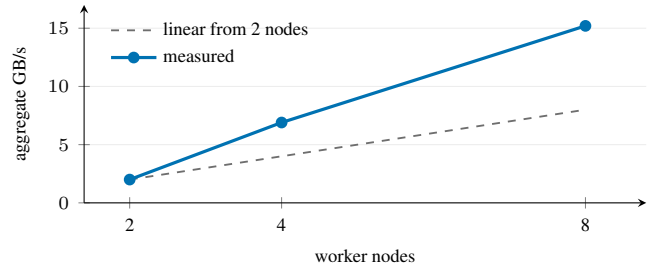


Figure 6: Aggregate all-to-all shuffle throughput against cluster width, with a linear reference anchored at two nodes. Four times the hardware returns $7.6\times$ the throughput. The mechanism is the algebra: mappers publish pre-aggregated state and not rows, so a hierarchical combiner tree plus credit-based flow control keeps per-node memory bounded as the cluster widens, and adding nodes adds reduce capacity instead of contention. A *single* reducer stays NIC-bound at ~ 2.7 GB/s, so the gain is in the aggregate, where every node reduces at once.

on all seven cases it can express and takes 8 of 11 from DuckDB and 9 of 11 from Polars.

On ClickBench [6] (43 queries, correctness-gated) Batcher wins 43 of 43 against DuckDB on identical Arrow input, every query, by $0.00\text{--}0.42\times$, and 35 of 40 against Polars (the five losses are sub-3 ms queries where fixed overhead dominates). Against DuckDB’s *native* store it wins a minority: 15 clear wins, 9 within $\pm 10\%$, 19 clear losses. We report the split and not a win count because a fifth of the suite sits inside the noise a shared machine adds, and a run of ours on a quieter box put it at 26 of 43.

The attribution is unambiguous even where the count is not. The same engine on the same 43 queries wins *all* of them on identical Arrow input and a minority against a compressed store with zone maps: one variable, isolated. The residue is storage, not kernels.

14.4 The small case, which a warm benchmark never measures

Section 2’s first consequence is that a small query pays no distribution tax, and a suite reporting best-of- N warm timings cannot check it: it never sees a shape for the first time, and that is what a user issues constantly. Measured separately, with a fresh literal per iteration so the plan cache cannot hit: optimizing a never-seen single-table filter costs 1.52 ms and a filter $\rightarrow$ join $\rightarrow$ group-by 2.75 ms, against 0.002 ms for a repeated shape. End to end over twelve never-seen shapes at $n = 10^4$, Batcher runs at $0.80\times$ DuckDB on a cold filter and $0.27\times$ on a cold `count(*)` over one. DuckDB pays a first-query cost too, and ours is the smaller. This is where a Python control plane could have been disqualifying and is not: 375 rules over seven phases iterated to a fixpoint cost single-digit milliseconds because nothing in them is per-row.

14.5 Scaling out

Figure 6 is the scaling result. It is superlinear, not merely linear, and the reason is worth separating from the headline: at two nodes the shuffle does not saturate the available reduce capacity,

Baseline	sf1		sf10		Note
	geomean b/x	wins	geomean b/x	wins	
DuckDB (native compressed store)	1.02×	12/22	1.49×	3/22	compressed store
DuckDB (same Arrow input)	—	—	0.53×	21/22	like-for-like*
Polars [34]	0.55×	19/22	0.45×	17/22	
Daft [9]	0.48×	18/20	0.52×	13/20	wrong answers on q6, q15 [†]
Spark [1]	0.041×	22/22	0.074×	22/22	reads Parquet in timed region [‡]

Table 3: TPC-H, all 22 queries, correctness-gated, one 96-core node. b/x is $\text{Batcher}_{ms}/x_{ms}$: **below 1.00 means Batcher is faster**. Suite totals (ms), sf1 / sf10: Batcher 618 / 4,491; DuckDB 616 / 2,353; Polars 1,256 / 10,624; Daft 1,224 / 9,261; Spark 14,555 / 57,781. *Separate run: DuckDB executing the same zero-copy Arrow Batcher is given, not a compressed store it was allowed to build first; 4,453 vs 8,436 ms. [†]Daft covers the 20 queries it can express, raising on q21 and q22; Batcher agrees with DuckDB on 22 of 22 at both scales. [‡]Spark reads Parquet inside the timed region where the other four receive an in-memory Arrow handle, so its ratio is overstated by an unisolated scan cost. Ray Data is a streaming ML runtime without a relational optimizer, so a SQL suite is the wrong instrument for it; we evaluate it in Section 14.6, on the pipelines it is built for.

Operator case	DuckDB	Polars	PyArrow	Daft
groupby-sum	0.80	0.42	0.60	0.37
groupby-2key	0.62	0.40	0.54	0.22
global-sum	0.04	0.06	0.04	0.02
filter-count	0.07	0.03	0.00	0.03
join-agg	1.25	1.18	0.32	0.46
sort-limit	1.09	0.02	0.01	0.10
filter-project	0.79	1.10	0.06	0.49
window-rank	0.84	0.12	—	hangs
window-runsum	0.50	0.15	—	hangs
window-lag	0.71	0.04	—	hangs
window-sum-part	0.64	0.85	—	hangs
Batcher wins	8/11	9/11	7/7	7/7

Table 4: Operator mix at sf1, $\text{Batcher}_{ms}/x_{ms}$; below 1.00 is a Batcher win. All engines take the same in-memory Arrow input, so this carries none of Table 3’s scan asymmetry. PyArrow (Acero) has no window functions.

so part of the gain from 2 to 4 is recovered inefficiency. The claim we stand behind is the shape, that adding nodes adds throughput without a per-node memory penalty, not a promise of $1.9\times$ per doubling forever.

The single-query picture matches. On TPC-H sf10 q6 with every engine reading the same S3 Parquet and correctness-gated, Batcher over 64 partitions runs 224.5 ms against Daft on its Ray runner at 535.8 ms and DuckDB-on-Arrow single-node at 457.3 ms, and Daft returns the wrong answer described above. Across a pipeline suite at sf1/sf10/sf100 Batcher is ahead of Ray Data on every pipeline and leads Daft on joins ($1.7\text{--}2.2\times$), group-by and metadata-only counts.

14.6 GPU and ML workloads

These are the workloads Ray Data was built for, which makes it the right yardstick and the wrong thing to keep score against. The question worth answering is whether an engine that treats decode and inference as relational operators can hold its own against a runtime purpose-built for them. It can. On $8\times T4$ with real models: ResNet-50 batch inference $2.05\times$, image embeddings $1.98\times$, fractional-GPU packing $1.96\times$, video-clip inference $3.6\times$, training-data ingest $3.0\times$, multimodal JPEG-to-

GPU $1.3\text{--}2\times$.

Three figures are much larger, and should not be quoted as general execution speed: audio feature extraction $12.5\times$, LLM batch inference $11.1\times$, text embeddings $47\times$. They come from two scheduling mechanisms, not kernels. The session-scoped model pools of Section 5 amortize load across executions, so the ratio scales with how much of a job is model load and shrinks as jobs grow. And native in-engine preprocessing keeps mel-spectrogram and MFCC as Arrow kernels where the comparator runs a per-batch Python UDF. Neither is an architectural advantage; both are choices any runtime could adopt, and we expect the gap to close. The durable claim is the $2\text{--}3.5\times$ range, and what it shows is that putting AI operators inside a query engine costs nothing against a specialized runtime.

A single exception bounds all of it: on one maximally-large compute-bound job both engines saturate the same GPU at the same FLOPs, so the ratio is parity. Nothing here beats a specialized runtime at arithmetic, and nothing here claims to.

A note on the harness. These ratios are only meaningful if the comparator was configured properly, and ours initially was not: our adapter registered each table as a single block, which is Ray Data’s unit of parallelism, so a 6M-row table ran on one core of 96. That was our defect, not a property of Ray Data. Every number above is from the corrected adapter, which writes tables as untimed setup and reads them back through the Parquet connector with row groups sized to Ray’s own target block size.

15 Related work

Batcher’s execution is morsel-driven [18] over a vectorized Arrow representation in the MonetDB/X100 lineage [5], with a compiled path for scalar expressions restricted to a subset carrying a proven-identical fallback. Volcano [11] established the operator/exchange separation; the departure is that Batcher has no distributed operator to separate.

Mid-query re-optimization [14], progressive optimization [23] and eddies [3] are the lineage of the stage-boundary clock, surveyed in [8], with Spark AQE [29] its closest deployed relative. Learned optimizers [21, 22] attack estimation with models; Batcher takes the lighter route of sketches [10, 15], refit

cost coefficients and a bandit [2] over result-equivalent strategies, which buys reproducibility and a cold start that is exactly the untuned system. [19] is the standing argument for why measured cardinalities beat better estimators, and Structured Streaming’s framing of a stream as an unbounded table [36] is the one Section 3 adopts.

16 Conclusion

The machine inverted, arrival became continuous, the catalog stopped being trustworthy, the data stopped being tabular, the model moved into the middle of the pipeline, the hardware stopped being one kind of thing, and the language became Python. Seven shifts, none of them gradual, and the engines most teams run predate all seven.

The workarounds are so familiar by now that they read as the cost of doing business. A warm cluster, so that small queries are bearable. A second execution model for anything fresher than a day. A shuffle width tuned by hand, and a CPU-to-GPU worker ratio picked the same way. A per-row callback for whatever SQL cannot say. And a duplicate of the data for whoever trains the models.

None of it is necessary, and one constraint retires all four boundaries at once. Admitting an operator only as a mergeable triple removes the split between the fast engine and the scalable one, so the same code runs on a laptop and on a cluster: speed is not re-earned at scale, and scale is not paid for in advance. Because that triple is also the incremental form, it removes the split between batch and streaming, and a query keeps its memory bound whether it reads a hundred terabytes of Parquet or a Kafka topic. Because the operator is identical everywhere, a measurement taken anywhere is valid everywhere, which is what lets the optimizer plan from evidence instead of from constants chosen years ago for hardware nobody is running. And because decode, embedding, vector search and inference are expressions in that same algebra over the same Arrow batches, the pipeline that turns raw bytes into model input is one optimized plan instead of three systems and two copies of the data.

The measurements bear that out on every axis, not one. Parity with the fastest single-node engine on small data and a clean sweep of it on equal input; superlinear shuffle scaling across eight nodes; constant memory on a stream where a materializing engine grows without bound; $2\text{--}3.5\times$ against a purpose-built GPU runtime on the pipelines it was designed for; and thirty formats and four model tiers reachable from one plan. One algebra did all of it, and that is the argument: the breadth is not four features bolted together but one decision, taken early and never broken.

Reproducibility. Every number comes from a committed harness: `benchmarks/run.py` drives the engine adapters and `harness.py` enforces the correctness gate, refusing to time any query whose result does not match DuckDB. The harness records the engine build profile with each run, so a result cannot silently be attributed to a different build. Timings are best-of- N warm on the machine described in Section 14, and every ratio in this paper is reproducible from that harness.

References

- [1] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. Spark SQL: Relational data processing in Spark. In *ACM SIGMOD International Conference on Management of Data*, 2015.
- [2] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [3] Ron Avnur and Joseph M. Hellerstein. Eddies: Continuously adaptive query processing. In *ACM SIGMOD International Conference on Management of Data*, 2000.
- [4] John Black and Phillip Rogaway. Ciphers with arbitrary finite domains. In *Topics in Cryptology — CT-RSA*, 2002.
- [5] Peter A. Boncz, Marcin Zukowski, and Niels Nes. MonetDB/X100: Hyper-pipelining query execution. In *Conference on Innovative Data Systems Research (CIDR)*, 2005.
- [6] ClickHouse, Inc. ClickBench: A benchmark for analytical databases, 2024.
- [7] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.
- [8] Amol Deshpande, Zachary Ives, and Vijayshankar Raman. Adaptive query processing. *Foundations and Trends in Databases*, 1(1):1–140, 2007.
- [9] Eventual Computing. Daft: A distributed query engine for large-scale multimodal data, 2024.
- [10] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Conference on Analysis of Algorithms (AofA)*, 2007.
- [11] Goetz Graefe. Volcano—an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994.
- [12] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data Mining and Knowledge Discovery*, 1(1):29–53, 1997.
- [13] International Data Corporation. Worldwide global DataSphere structured and unstructured data forecast, 2025–2029. Technical Report US52800025, International Data Corporation, 2025.
- [14] Navin Kabra and David J. DeWitt. Efficient mid-query re-optimization of sub-optimal query execution plans. In *ACM SIGMOD International Conference on Management of Data*, 1998.
- [15] Zohar Karnin, Kevin Lang, and Edo Liberty. Optimal quantile approximation in streams. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2016.
- [16] Zuhair Khayyat, William Lucia, Meghna Singh, Mourad Ouzzani, Paolo Papotti, Jorge-Arnulfo Quiáné-Ruiz, Nan Tang, and Panos Kalnis. Lightning fast and space efficient inequality joins. *Proceedings of the VLDB Endowment*, 8(13), 2015.
- [17] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *Conference on Innovative Data Systems Research (CIDR)*, 2019.
- [18] Viktor Leis, Peter Boncz, Alfons Kemper, and Thomas Neumann. Morsel-driven parallelism: A NUMA-aware query evaluation framework for the many-core age. In *ACM SIGMOD International Conference on Management of Data*, 2014.
- [19] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. How good are query optimizers, really? *Proceedings of the VLDB Endowment*, 9(3):204–215, 2015.
- [20] Michael Luby and Charles Rackoff. How to construct pseudorandom permutations from pseudorandom functions. *SIAM Journal on Computing*, 17(2):373–386, 1988.
- [21] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. Bao: Making learned query optimization practical. In *ACM SIGMOD International Conference on Management of Data*, 2021.
- [22] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. Neo: A learned query optimizer. *Proceedings of the VLDB Endowment*, 12(11), 2019.
- [23] Volker Markl, Vijayshankar Raman, David Simmen, Guy Lohman, Hamid Pirahesh, and Miso Cilimdžić. Robust query processing through progressive optimization. In *ACM SIGMOD International Conference on Management of Data*, 2004.
- [24] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
- [25] MosaicML / Databricks. StreamingDataset: Fast, accurate streaming of training data from cloud storage, 2023.
- [26] Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *ACM SIGMOD International Conference on Management of Data*, 2019. Demonstration.
- [27] David Reinsel, John Gantz, and John Rydning. The digitization of the world: From edge to core. IDC White Paper US44413318, International Data Corporation, November 2018.
- [28] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *ACM SIGMOD International Conference on Management of Data*, 1979.
- [29] The Apache Software Foundation. Adaptive query execution — Spark SQL performance tuning guide, 2024.
- [30] The Apache Software Foundation. Apache Arrow: A cross-language development platform for in-memory analytics, 2024.
- [31] The Apache Software Foundation. Arrow Flight RPC, 2024.

- [32] The Ray Team. Ray data: Scalable datasets for ML, 2024.
- [33] Transaction Processing Performance Council. TPC Benchmark H standard specification, 2022.
- [34] Ritchie Vink and Polars contributors. Polars: DataFrames for the new era, 2024.
- [35] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.
- [36] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized streams: Fault-tolerant streaming computation at scale. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2013.